# CSE 242 — Homework 6 Report

*Free-Form Trading on the lu-eth FakeETH Market*

**Author:** foj227@lehigh.edu **Team:** jfj **Date:** October 28–29, 2026 **AI assistance:** Claude (Anthropic), used per H6 §5. All strategy, code, and analysis below were produced with that assistance and are documented here and in the accompanying files. Use of an AI assistant and the intent to exploit buggy pools were cleared with the instructional team beforehand.

---

## 1. Executive summary

I approached the market first as a trader (looking for arbitrage) and then as an attacker (looking for exploitable bugs in classmates' exchanges, which §5 permits). To do this safely and at scale I built three tools: a **read-only market scanner**, an **atomic arbitrage contract**, and an **EVM bytecode analyzer**.

The headline finding is that **this market is efficient and the exchanges are well-built.** Every live pool trades each token at the same price, and because the AMMs charge no fee, that price parity leaves no arbitrage. I audited the deployed bytecode of **every exchange implementation holding meaningful FakeETH (~6 FakeETH across four distinct codebases)** and found all of them secure against the classic AMM vulnerabilities — reentrancy, `msg.value` double-counting, missing ratio checks, and pool-draining. The one pool that *appeared* badly mispriced turned out not to be an exchange at all, but a classmate's trading bot holding leftover inventory.

My conclusion is therefore a defensive one: there is no profitable attack or arbitrage available, so the correct play is capital preservation. I kept the atomic arbitrage contract deployed as a zero-risk tool that will capture any imbalance *if* one ever appears, since by construction it cannot lose principal.

**Net result:** holdings essentially unchanged (one test transaction that safely reverted, costing only negligible gas). Details in §7.

---

## 2. Tools built

All three are included with this submission.

**`scan_exchanges.js`** — a read-only script run inside Remix against lu-eth. For every exchange address it reads the ETH and ERC-20 reserves, computes the implied price, calls each `estimate*` view, compares the quotes against the fair constant-product value, groups pools by shared bytecode hash, and reports the price spread per token. It sends no transactions and needs no approvals. This is what turned a list of addresses into a picture of the whole market in one pass.

**`ArbBot.sol`** — an atomic two-leg arbitrage contract. Given a "buy" exchange and a "sell" exchange, it buys a token on one and sells it on the other **in a single transaction**, and reverts unless it ends with at least a caller-specified minimum profit in FakeETH. This defeats the timing risk the handout describes (a third party trading between your two legs): the worst case is a reverted transaction costing only gas, never principal. It also only ever grants *exact* token approvals to another contract, never an unlimited allowance.

**`analyzer_evm.py`** — a self-contained EVM disassembler (including a from-scratch keccak-256, since the sandbox had no crypto library) that maps a pool's bytecode to the H4 interface functions and flags the tell-tale "external call before state update" pattern that indicates reentrancy. This is what let me audit unverified bytecode directly.

---

## 3. Market reconnaissance

Using the block explorer's token-holder API I enumerated every contract holding ASA, HAW, or KOR — 22 exchange addresses plus our own three. The scanner then pulled live reserves for all of them.

**The market is at near-perfect price parity.** Within each token every working pool quotes the same price to 7–8 significant figures:

| Token | Price (FakeETH per token) | # live pools |
|---|---|---|
| ASA | 0.00516198 | 6 |
| HAW | 0.00674791 | 8 |
| KOR | 0.00635024 | 7 |

Because the AMMs charge **no swap fee**, once prices are equal across pools there is no arbitrage to capture — any round trip returns exactly what you put in, minus slippage. There is also no triangular arbitrage, because the only trading pairs are token/ETH (there is no ASA/HAW market, etc.). The scanner's estimate-vs-fair-value check additionally found **no "lying" pools** whose quotes deviated from the honest constant-product value.

*3.1 The one apparent anomaly — and what it really was*

A single address, `0x6fA0…5a4c`, showed HAW priced ~97% below market and reverted on every `estimate` call. It looked like either a broken pool to drain or a mispriced one to arbitrage. I examined its bytecode and found it is **not an exchange at all**: it has an `owner`, `"owner only"` guards, and helper functions that call `swapForEth` / `provideLiquidity` / `withdrawLiquidity` on *other* pools (one runs a provide-then-withdraw loop). It is a classmate's **trading bot**, holding 3.36 HAW and 0.0006 FakeETH as leftover inventory — not AMM reserves. That is why it doesn't implement the exchange interface (hence the reverts) and why the "price" was meaningless. Its funds are owner-locked, so there was nothing to take.

---

## 4. The arbitrage attempt (and why "failure" was the right outcome)

Before realizing `0x6fA0` was a bot, I routed a small trade through `ArbBot` to buy its "cheap" HAW and sell on a deep pool. The transaction **reverted inside `0x6fA0`'s swap call** and cost only gas. This is exactly the behavior the contract is designed for: the bad leg reverted the whole atomic transaction, so no principal was ever at risk. Had I done this as two manual transactions instead, I could have sent real FakeETH into a contract that doesn't return anything. **The atomic design converted a potential loss into a rounding-error gas cost** — the key safety lesson of this assignment, and a direct application of the H6 warning about trades landing between your two legs.

---

## 5. Security audit of the exchanges

Since no arbitrage existed, the only path to profit was exploiting a buggy exchange (§5). I disassembled the deployed bytecode of every implementation that holds meaningful FakeETH. Pools cluster by bytecode hash; each cluster is one team's implementation (and therefore shares any bug). Four distinct implementations cover ~6 FakeETH:

| Implementation | Pools | FakeETH | Reentrancy defense | Verdict |
|---|---|---|---|---|
| A (`0x993b5a`) | 3 (ASA/HAW/KOR) | ~2.34 | `ReentrancyGuard` | secure |
| B (`0x7dae7b`) | 3 (ASA/HAW/KOR) | ~2.34 | none — but CEI-correct | secure |
| C (`0xe594d7`*) | 3 (ASA/HAW/KOR) | ~0.97 | boolean guard | secure |
| D (`0x76168d`) | 3 (ASA/HAW/KOR) | ~0.24 | `ReentrancyGuard` | secure |

| | | | | |
|---|---|---|---|---|
| | | | | |

* Implementation C compiles the same source three times with a different immutable token address, so its three pools (`0x311E`, `0x35b5`, `0x800C`) have different bytecode hashes (`0xe594d7` / `0x732734` / `0xbe2064`) but are one codebase.

I checked each implementation for the four vulnerabilities most likely to be present:

**Reentrancy (the DAO bug H4 warns about).** The profitable version is re-entering `withdrawLiquidity` (or a swap) during the ETH payout, before the contract decrements your position. Implementations A, C, and D have an explicit reentrancy guard. Implementation B has **no guard**, but it is still safe because it follows checks-effects-interactions: it writes `liquidityPositions`, `totalLiquidityPositions`, and `K` to storage *before* transferring tokens or sending ETH, so a re-entrant call sees the already-updated state and gains nothing. In every implementation the ETH transfer is the last action.

**`msg.value` double-counting.** The subtle AMM trap (the handout's "Ether is NOT an ERC-20 token" hint): in the payable `swapForERC20Token`, the sent ETH is *already* in `address(this).balance`, so the correct post-swap balance is `SELFBALANCE` — not `SELFBALANCE + msg.value`. All four implementations compute `tokenOut = tokenBalance - K / SELFBALANCE`, which is correct. None double-counts, so none can be made to hand out tokens cheaply. (My scanner only checks the *view* estimates, which take a parameter instead of `msg.value`; I confirmed the payable path by hand-tracing the bytecode of each implementation.)

**Missing / loose ratio check in `provideLiquidity`.** If a pool didn't enforce that deposited ETH and tokens match the current reserve ratio, you could deposit lopsidedly and withdraw more than you put in. All four enforce the ratio — B with a ±1-unit rounding tolerance, C and D with exact cross-multiplied equality.

**Pool-draining via `withdrawLiquidity`.** All four reject burning the entire pool (`"Cannot withdraw all liquidity"` / `amount < totalLiquidityPositions`) and reject burning more positions than you own. Implementation D additionally bounds every swap output to `0 < out < reserve` and recomputes `K` locally each call rather than trusting a cached value — the most defensive of the set.

The only pools left unaudited are three dust singletons (`0xd477` at 0.076, `0x592A` at 0.023, and an empty pool), whose combined value is negligible and not worth the gas to attack even if flawed.

---

## 6. Why everything was secure (discussion)

The most interesting result of this assignment is a negative one. The rules invite exploitation, but in practice the market was watertight. I think the explanation is exactly what the handout hints at: everyone had access to the same AI assistance, so the common pitfalls — reentrancy, `msg.value` confusion, unchecked ratios — were caught before deployment. The remaining differentiator between a good submission and a bad one is no longer "did you leave the obvious bug in," but "did you reason carefully about where the bug *would* be, and prove it isn't there." That reasoning is the substance of §5 and the point of this report.

---

## 7. Holdings

| | FakeETH | ASA | HAW | KOR |
|---|---|---|---|---|
| Before | 9.988 | 0 | 0 | 0 |
| After | ≈9.988 (−negligible gas) | 0 | 0 | 0 |

No position was opened because none was profitable. The one transaction sent (the `0x6fA0` arbitrage attempt) reverted atomically by design and moved no principal. `ArbBot` remains deployed at `0xAde3f9f0C4B4aDF9aDb7057b4f5ACeebA7724710` and funded with 0.01

FakeETH as a standing, risk-free tool.

---

## 8. Files included

- `H6_report.pdf` — this document.
- `H6_market_log.md` — the chronological working log (every finding, every transaction, with reasoning).
- `ArbBot.sol` — the atomic arbitrage contract.
- `scan_exchanges.js` — the read-only market scanner.
- `analyzer_evm.py` — the EVM bytecode analyzer used for the security audit.
- `scan_output.json` — the live market snapshot the analysis is based on.

---

## 9. Appendix — key transactions

| Action | Address / tx | Result |
|---|---|---|
| Deploy `ArbBot` | `0xAde3f9f0C4B4aDF9aDb7057b4f5ACeebA7724710` | success |
| Fund `ArbBot` 0.01 | tx `0xc795…95c6` | success |
| Arbitrage attempt vs `0x6fA0` | tx `0x4eec…0628` | **reverted (by design, no loss)** |